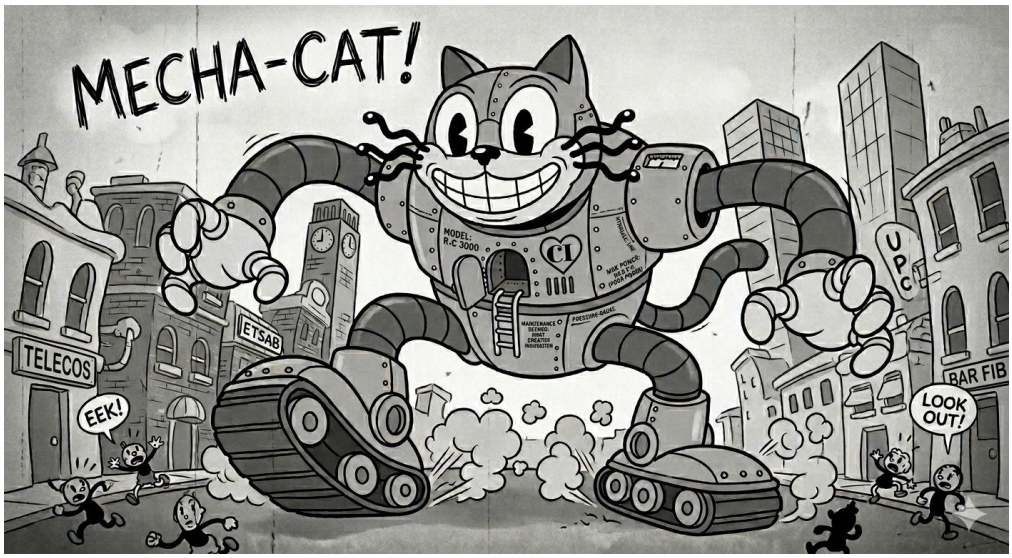
Computer Interfacing (CI) FIB. **Primer parcial** **10/04/2026**

Cognoms, Nom _____ **DNI/NIE**_____

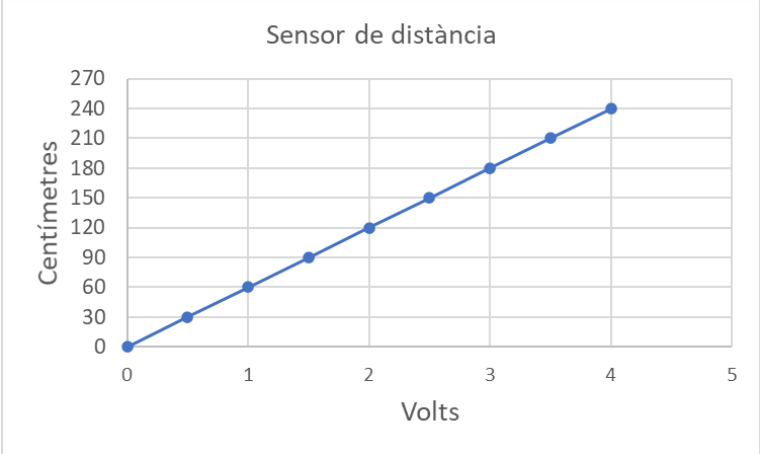
Tota resposta sense justificar es considerarà nul·la !

P1. (4 punts) Els professors de CI finalment hem decidit crear el nostre propi gat *mecha* (robot gegant). Per fer-ho realitat i obtenir l'ajuda dels alumnes de CI hem triat dissenyar diferents parts del nostre *mecha* fent servir el PIC18F45K22 (amb Fosc=10MHz). Per exemple, a cada peu posarem un sensor de distància per saber si estem recolzant al terra o a quina distància el tenim.



Nota: Imatge generada amb IA a efectes il·lustratius

1.1 (1p) Quina és la resolució en cm que obtindrem amb el nostre AD de 10 bits si fem servir Vref+=5V i Vref-=0V? Com pots millorar aquesta resolució?



Amb un ADC de 10 bits i Vref+ = 5V, Vref- = 0V:

Resolució:

$$\frac{V_{ref+} - V_{ref-}}{2^{10}} = 4.883mV/ticAD$$

Com que el sensor té una resposta lineal, el seu factor de conversió és:

$$\frac{240cm}{4V} = 60cm/v$$

Obtenim una resolució de:

4.883mV*60cm/V = **2,93mm / ticAD**

Per millorar aquesta resolució tenim diverses estratègies, com per exemple:

Ajustar les tensions de referència al rang útil del sensor: El PIC18F45K22 permet seleccionar com a Vref+ i Vref- els pins externs **AN3/Vref+** i **AN2/Vref-** o fer servir el generador de tensions de referència, en lloc de fer servir VDD i VSS.

Amplificar el senyal del sensor per aconseguir que s'ajusti més als valors de referència del nostre AD

Fer servir un AD amb més bits. Però això ja implica canviar de microcontrolador o fer servir un AD extern.

1.2 (1,5p) Si fem servir un únic PIC per llegir els dos peus (un sensor a cada peu), cada quant tindrem una lectura del peu dret? Quina és la freqüència màxima que podem digitalitzar sense patir aliasing? (TAD≥1µs i TACQ≥7,5µs)

Amb Fosc=10MHZ, el període d'instrucció és: Tcy=4/Fosc=0,4 µs.

Primer seleccionarem el TAD:

TAD = 2·TCY = 0,8 µs -> no compleix condicions

TAD = 4·TCY = 1,6 µs -> compleix condicions

Després seleccionem el TACQ entre les opcions possibles: 2, 4, 6, 8, 12, 16 o 20 TAD. El mínim que podem aconseguir seguint les limitacions de l'enunciat és 6TAD -> TACQ=6*1,6µs=9,6µs.

Ara ja podem calcular el temps per obtenir una mostra = TACQ + 12 TAD = 9,6µs + 12*1,6µs=28,8µs.

Com que la lectura dels diferents canals d'AD està multiplexat, haurem de llegir de forma alterna el peu dret i el peu esquerre, així que tindrem una mesura del **peu dret cada 57,6µs, que és un mostreig de 17,36KHz.**

Aplicant el criteri de Nyquist (la freqüència de mostreig ha de ser com a mínim el doble de la freqüència del senyal):

Fmax=8,68KHz

1.3 (0,75p) Un grup vol implementar la lectura de l'AD fent servir interrupcions. Indica quins bits has de configurar si vols obtenir una interrupció d'AD sense prioritats

La configuració és:

```
RCON.IPEN  = 0;   // Sense prioritats
PIR1.ADIF  = 0;   // Netegem el flag per seguretat
PIE1.ADIE  = 1;   // Habilitem la interrupció de l'AD
INTCON.PEIE = 1;  // Habilitem les interrupcions de perifèrics
INTCON.GIE  = 1;  // Habilitem les interrupcions globals
```

1.4 (0,75p) Un altre grup vol que cada vegada que volem llegir l'AD ho fem mitjançant enquesta periòdica del bit GODONE. Creus que té sentit fer-ho?

El bit GO/DONE té un comportament molt concret:

L'escrivim a 1 per iniciar una conversió.
El maquinari el manté a 1 mentre la conversió està en curs.
El maquinari el posa automàticament a 0 quan la conversió acaba, i en aquell moment el resultat ja és vàlid als registres ADRESH:ADRESL.

Si fem un codi com:

```
 GODONE = 1;              // Iniciem la conversió
while (GODONE);     // Esperem fins que acabi
resultat = (ADRESH << 8) | ADRESL;
```

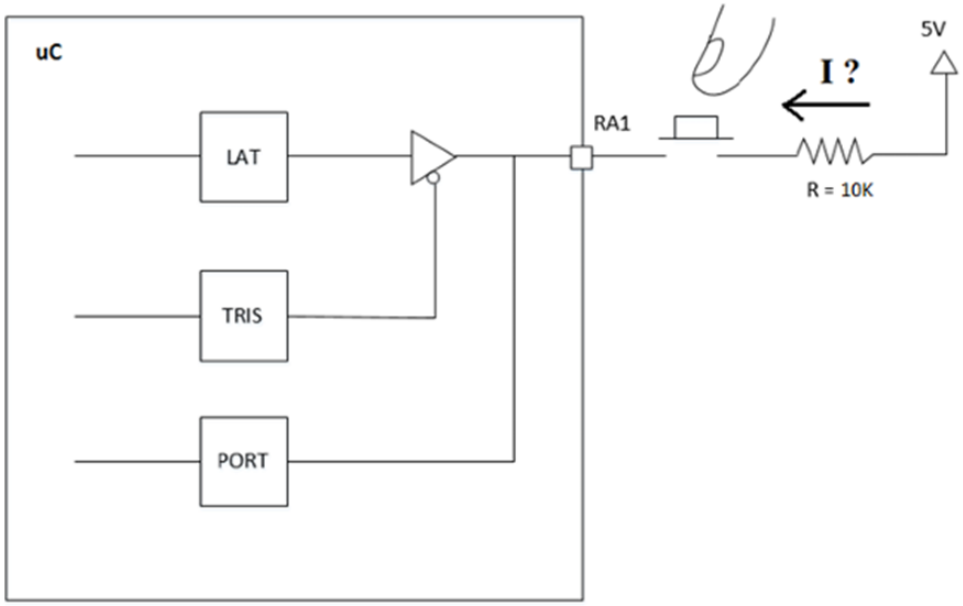
El temps que estarem bloquejats a l'espera del bucle while son uns 28,8µs. A partir d'aquest punt depèn del temps de latència de la interrupció (3-4TCY) i del temps de servei a l'RSI. Les dues argumentacions es poden donar per vàlides si estan ben raonades.

Computer Interfacing (CI) FIB. Primer parcial **10/04/2026**

Cognoms, Nom _____ **DNI/NIE**_____

Tota resposta sense justificar es considerarà nul·la !

P2. (1.5 punts) En la següent figura teniu una representació esquemàtica simplificada d'un port d'E/S. Concretament d'un dels seus bits (el bit 1 del Port A). A l'exterior del micro, hem posat un circuit molt bàsic, amb un polsador, una resistència (10KOhms) i una connexió a alimentació (5V).



Executem el següent codi:

TRISA=0x33

LATA=0x55

Indica de forma justificada que llegirem a:

Bit 1 del PORTA si el botó està premut: Hem configurat el pin com a input. Al prémer el botó llegirem, RA1 es connecta a VDD → Valor lògic = 1.

Bit 1 del LATA si el botó està premut: Com que RA1 és entrada (TRISA[1]=1), el valor de LATA[1] no afecta la sortida física (tristate en alta impedància). LATA[1] queda escrit amb un 0 per LATA=0x55.

Bit 1 del PORTA si el botó no està premut: No hi ha cap resistència de pull connectada, com que el pin és de input, estarà a l'aire, amb un valor indeterminat.

Circula corrent pel circuit quan el botó està premut? Quants ampers? El pin RA1 és d'entrada (alta impedancia) i no existeix camí tancat per fer circular corrent significatiu. Com a màxim hi haurà un Input Leakage de 50nA

P3. (3 punts) Fent servir Claude de l'empresa Anthropic, hem generat el codi que trobareu a continuació. Nosaltres, sabem que el contingut de la memòria de dades del PIC18 en el que s'executarà conté els registres que trobareu a la taula adjunta.

```
      count set 0x00
      n set 0x01
      threshold equ D'100'
      MOVLB   0x05
      CLRF    count, A
      MOVLW   D'16'
      MOVWF   n, A
      LFSR    FSR0, 0x300

CountIf
      MOVLW   threshold
      CPFSGT  POSTINC0
      BRA     Skip
      INCF    count, F, A
Skip
      DECFSZ  n, F, A
      GOTO    CountIf
```

Registre	Valor	Registre	Valor
0x000	0	0x308	0x99
0x001	0	0x309	0x4D
0x300	0x3A	0x30A	0x60
0x301	0x7F	0x30B	0xB3
0x302	0x12	0x30C	0x1E
0x303	0xC4	0x30D	0x77
0x304	0x55	0X30E	0x44
0x305	0x08	0x30F	0x64
0x306	0xA1	0x310	0xFF
0x307	0x2B	BSR	5

3.1 (0,75p) Explica, de forma breu, que fa aquest codi.

Aquest codi compte quants registres tenen un valor més gran que 100 (decimal) en els primer 16 registres del Bank3. Guarda aquest resultat al registre 00 de l'access bank

3.2 (1,5p) Si Fosc=4Mhz, quant triga en executar-se? El temps d'execució depèn del contingut del bank 3 de la memòria de dades?

Veiem que abans del bucle triga 1c (MOVLB)+1c(CLRF)+1c(MOVLW)+1c(MOVWF)+2c(LFSR)=6c

Fa 15 iteraciones al bucle:

si CPFSGT salta: 1c(MOVLW)+2c(CPFSGT)+1c(INCF)+1c(DECFSZ)+2c(GOTO)=7cicles
si CPFSGT no salta: 1c(MOVLW)+1c(CPFSGT)+2c(BRA)+1c(DECFSZ)+2c(GOTO) =7cicles

Una útlima iteració com:
si CPFSGT salta: 1c(MOVLW)+2c(CPFSGT)+1c(INCF)+3c(DECFSZ)=7cicles
si CPFSGT no salta: 1c(MOVLW)+1c(CPFSGT)+2c(BRA)+3c(DECFSZ) =7cicles

Aquí veiem que el temps d'execució **no depèn del contingut del bank3.**

En total triga 6c+16*7c=118 cicles. Com que Fosc=4MHz,TCY=1µs. **Triga 118 µs.**

3.3 (0,75p) Dels registres de la taula, indica quins es veuen modificats (escriu el seu valor final).

Dels registres de la taula, es modifica 0x000, que acaba amb el valor 6 (el registre 0x310 és el 17è registre i no es comprova).

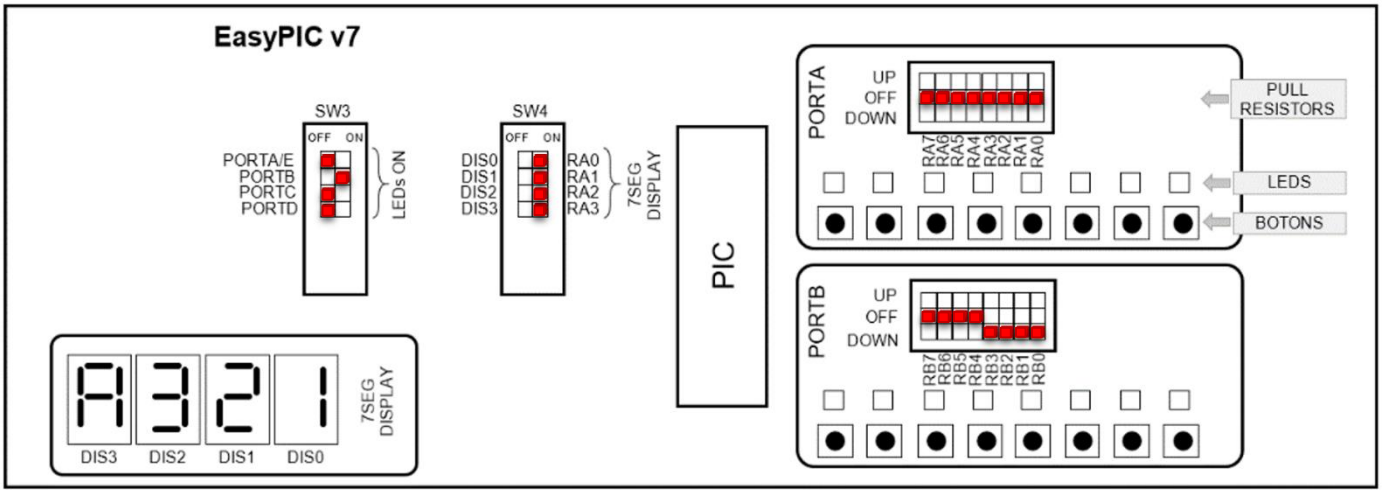
0x001, que s'inicialitza a 16 però que acaba amb valor=0 i BSR,que s'escriu amb el mateix valor que té a l'inici del codi (5).

Computer Interfacing (CI) FIB. Primer parcial **10/04/2026**

Cognoms, Nom _____ **DNI/NIE**_____

Tota resposta sense justificar es considerarà nul·la !

P4. (1,5 punts) Volem emular un comandament de TV usant la placa EasyPIC v7. De moment implementarem els botons de pujar i baixar de canal i de pujar i baixar el volum de l'altaveu. El display de 7 segments mostra un valor de canal o de volum, e.g. 'C555' (canal 555), 'A321' (altaveu 321).
Recordeu que els nostres polsadors van connectats a Vdd, que és el valor que obtenim en prémer-los.
A la imatge teniu dibuixats alguns dels interruptors de la placa sense cap configuració. Haureu de dibuixar les posicions dels interruptors SW3, SW4 i PULL-Resistors, segons el que demanin els següents apartats.



4.1 (0,5p) Els comandaments usen un led IR (infrarrojos) per emetre les ordres a la TV. En la nostra placa usarem un led normal per simular aquesta funció, el led RB7. Per pujar canal detectarem el flanc ascendent en el botó RB3, baixar canal serà amb flanc ascendent en botó RB2. Per la funció de pujar volum llegirem per nivell el polsador RB1 (mentre el nivell és 1, enviem petició de pujar contínuament). Anàlogament per baixar volum, llegim per nivell el polsador RB0.

Marca sobre la imatge la posició com deixaries TOTS els interruptors de la placa que hi apareixen, i raona aquí la seva justificació.

Apareixen 24 interruptors (SW3, SW4, Pulls A i Pulls B). Els deixem de la següent manera:

A SW3 activem el de PORTB per poder visualitzar el led RB7. Dels altres ports no encendrem cap led, per tant podem deixar la resta de SW3 en off.

A SW4 hem d'activar els quatre interruptors DISx per connectar-los als pins RAx, i així poder enviar les senyals de control de multiplexació als displays 7seg.

Als Pulls de PORTA, els deixem tots en OFF doncs no fem lectura de cap polsador, que requereixi pull-down. Les senyals de control dels displays son de sortida i per tant no requereixen pull-down.

Als Pulls de PORTB, connectem en Pull-Down els pins RB0, RB1, RB2 i RB3 doncs farem lectura de polsadors. Tal com diu l'enunciat tots els polsadors premuts donen Vdd, per tant quan no es premen han de donar 0V i requereixen posar un Pull-Down (no importa si llegim per nivell, o flanc ascendent o flanc descendent). Els altres pins deixem els Pulls en OFF doncs ja no hi ha cap altre polsador llegit en aquests pins (i el Led RB7 és pin de sortida).

4.2 (1p) Ens hem quedat adormits amb el dit polsant el botó de baixar volum. Això vol dir que cada iteració del bucle principal, a part de mostrar els 4 displays 7seg, detecta el nivell 1 del botó RB0 i envia la petició a la TV usant el led RB7.

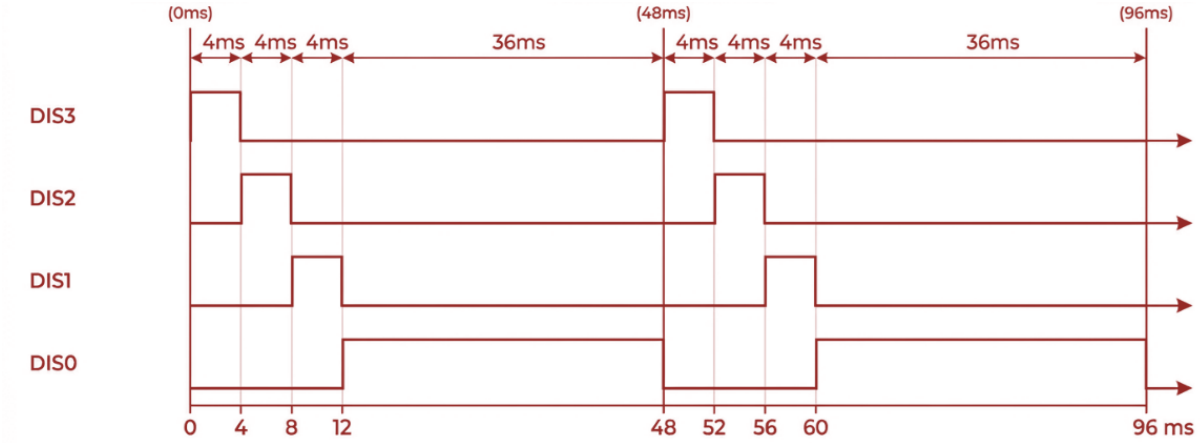
La petició consisteix en fer aquesta seqüència al led: ENCÈS 16ms, APAGAT 8ms, ENCÈS 8ms i finalment l'apaguem.

Mentre mantinguem el botó premut, els quatre símbols del display 7seg es veuran correctament?
O es produiran efectes visuals indesitjats?
Justifica la teva resposta.

```
while(1) {  // bucle principal
  ...  // deteccio pujar canal
  ...  // deteccio baixar canal
  ...  // deteccio pujar volum
  if (RB0==1) {  // deteccio baixar volum
    if (valorVolum>0) valorVolum--;
    // envia peticio -1 a volum TV
    encenLedRB7(); __delay_ms(16);
    apagaLedRB7(); __delay_ms(8);
    encenLedRB7(); __delay_ms(8);
    apagaLedRB7();
  }

  //Multiplexacio Displays
  apagaTot(); mostraDIS3(); __delay_ms(4);
  apagaTot(); mostraDIS2(); __delay_ms(4);
  apagaTot(); mostraDIS1(); __delay_ms(4);
  apagaTot(); mostraDIS0(); __delay_ms(4);
}
```

La multiplexació en el temps de les 4 xifres 7seg serà com es veu a la figura següent. Després de mostrar el DIS0 fa un delay de 4ms, però no l'apaga! sino que al fer la següent iteració del while(1) i detectar pulsació de RB0, enviarà la petició a la TV que comportarà els delays de 16+8+8ms = 32ms.



Les instruccions de apagaTot() abans de fer el canvi de xifra protegeixen contra el Ghosting.
El temps d'execució de la resta del codi no el tenim en compte doncs serà negligible en comparació amb els delays de milisegons.

Sí que es produiran efectes visuals indesitjats, s'accepta com a resposta qualsevol d'aquests dos:

1) la xifra DIS0 està encesa molt més temps per cicle que les altres tres. Per tant es veurà il·luminada molt més intensament, mentre que les altres tres xifres es veuran molt tènues.

2) cada xifra s'encén i s'apaga amb un període de 48ms, això és una freqüència de parpadeig de 20.8Hz, molt baixa, i per tant percebrem Flickering.